

MatQuery: Execution-Grounded Natural-Language Querying over Heterogeneous Materials Databases

Chen Jason Zhang and Gang Li

*The Hong Kong Polytechnic University
Hung Hom, Kowloon, Hong Kong*

Technical Report

ABSTRACT

Materials scientists increasingly combine information from several public databases, each with its own interface, schema, units and calculation conventions. MatQuery is a prototype and experimental infrastructure for studying how a natural-language request can be turned into queries over such databases and how the returned values can be interpreted. This report describes what has been built and the preliminary results obtained so far. The prototype holds frozen, self-verifying local copies of six public materials databases (about 10,000 records each), a catalog of declared fields derived from them, a database-independent query representation with an adapter for each database, and an eight-step pipeline in which a language model proposes and deterministic program steps check, execute and assemble. A benchmark generator produces questions whose reference queries are verified by execution on the frozen copies and whose wording is checked by back-translation and review. An evaluation harness runs direct prompting, CHESSE and Alpha-SQL on the same frozen basis. Preliminary experiments on an 800-question development set show that direct generation with three demonstrations reaches 82.4% strict execution accuracy (56.5% without demonstrations), that a low-cost scorer selects the exact set of databases for 85.0% of complete requests with well-calibrated probabilities, and that band gaps differ by more than 0.5 eV in 31.6% of composition-matched cross-database clusters. Two pilots show the limits of the current approach: a probe-then-ask policy gave answers rated valid by both reviewers in only 11 of 50 sessions, and only 115 of 300 sampled record pairs could be compared by structure. The complete pipeline is in smoke testing and has not yet been evaluated at scale.

KEYWORDS

Natural-language interfaces to databases; text-to-SQL; heterogeneous data integration; query federation; materials databases; benchmark construction; data provenance.

1 INTRODUCTION

A materials scientist who screens candidates for a new compound rarely finds everything in one place. The Materials Project [11], OQMD [13], AFLOW [4] and JARVIS-DFT [3] publish properties computed by density-functional theory, NOMAD [5] archives the raw output of many such calculations, and the Crystallography Open Database (COD) [8] collects crystal structures determined by experiment. Each database answers its own kind of question well. A request such as “which of these candidates have actually been made, and how do their computed band gaps compare across databases?” nevertheless requires several of them at once, and the scientist must still find the right fields, align their units, and decide whether two records describe the same material.

Natural-language interfaces to databases [14] promise to remove much of this work, and agents such as LLaMP [2] and ChatMOF [12] already ground a language model in one materials source. Recent text-to-SQL systems based on large language models perform well on benchmarks such as Spider [20] and BIRD [15], using decomposition [18], prompt design [7], schema pruning [19] or search [16]. Those benchmarks, however, involve one relational database at a time and a question whose meaning is complete. A materials request differs in three ways. It may need several databases whose schemas share no column names. Its scientific meaning depends on conventions, such as the calculation method behind a band gap, that the request often leaves unstated. And a query that executes correctly does not establish that the values it returns can be compared, because two records with the same formula may describe different crystal structures.

MatQuery was built to study these problems on real data. Its design follows one principle: the language model proposes, and the program checks. A model reads the request, chooses databases, matches fields and writes queries. Deterministic steps verify each proposal against a catalog of declared fields, run the query on a frozen local copy, recognize the same material across databases and assemble an answer table that keeps every value’s source. The same frozen copies serve a benchmark generator and an evaluation harness, so that every system is compared on an identical basis.

This report documents the prototype, the benchmark construction and the evaluation harness, and it reports the preliminary results obtained so far. Section 2 describes the data, Section 3 the pipeline, Section 4 the benchmark generator and Section 5 the evaluation harness. Section 6 reports the preliminary results, Section 7 the current status and the work that follows, and Section 8 the limits of the evidence. Section 9 summarizes the report.

Current status. The data layer, the field catalog, the query representation, the adapters, the eight-step pipeline, the benchmark generator and the evaluation harness are implemented. The complete pipeline is in smoke testing on a 30-sample validation profile, and its full-scale runs and ablations are in progress. The preliminary results in Section 6 were obtained on a development set of 800 questions with direct generation and with separate pilot configurations, not with the complete pipeline of Section 3. Independent expert review of the questions has not yet taken place, and the pilot labels were produced by AI reviewers and then checked by team members with materials-science backgrounds.

2 DATA: SIX DATABASES AND THEIR FROZEN COPIES

2.1 The six sources

The prototype draws on six public sources that differ in how they are accessed, how many fields they declare and what kind of data they hold (Table 1). Five hold computed data; NOMAD is an archive of calculations from many groups and codes, and COD holds experimental structures. Access ranges from an official Python SDK (Materials Project) and REST interfaces (OQMD, NOMAD) to the AFLUX query language (AFLOW), a downloadable release (JARVIS-DFT) and a public MySQL service (COD). The OPTIMADE specification [1, 6] offers a common query interface to several of these providers, but it standardizes how data are requested rather than what the returned values mean, so MatQuery keeps a source-specific adapter for each database.

Table 1: The six sources and their frozen copies in the current snapshot set. “Fields” counts the fields each source’s interface declares in the prototype’s catalog.

Source	Native access	Records	Fields	Kind of data
Materials Project [11]	Python SDK	10,000	33	computed
OQMD [13]	HTTP REST	10,000	23	computed
AFLOW [4]	AFLUX	10,000	37	computed
JARVIS-DFT [3]	JSON release	9,996	60	computed
NOMAD [5]	JSON over HTTP	10,000	28	computed (archive)
COD [8]	MySQL	10,000	40	experimental

2.2 Frozen, self-verifying snapshots

Every experiment runs on local copies rather than on the live services. A snapshot captures the state of one upstream database at one time as a read-only copy that verifies itself: it stores the normalized records, the raw payloads, a SQLite representation, summary statistics and a manifest with file digests, and validation checks the record counts, the digests and the consistency of the stored representations. Once a snapshot has been taken, all queries and evaluations read only from it and make no network call. Live services change, and a result obtained today could not otherwise be reproduced next month. The current snapshot set is identified as `materials-project-hard`, `oqmd-hard`, `afLOW-hard`, `jarvis-hard`, `nomad-hard` and `cod-hard`. The collection used for the preliminary results in Section 6 holds 10,000 records per source, selected by a deterministic diversity procedure with a fixed seed. Both collections are development resources selected for coverage, not samples of the complete databases.

A normalized record carries a namespaced identifier, the database and snapshot identifiers, the upstream record identifier, the formula and element set, structural descriptors, property values, method metadata, the data origin, the source URL and a digest of the raw payload. Namespacing prevents an identifier from one source from being confused with the same string in another. Scientific values remain nullable: a source that does not report a quantity is not given a substitute value, and in particular a missing band gap is never treated as zero. Method strings are kept as the source wrote them. Normalization therefore gives the query layer a common interface, while provenance keeps the path back to the original evidence.

2.3 Two kinds of heterogeneity

Asking one question across the six sources requires overcoming two differences. The first is naming. The band gap is called `Egap` in AFLOW, `band_gap` in the Materials Project and OQMD, `optb88vdw_bandgap` in JARVIS-DFT and a nested `...band_gap.value` in NOMAD, and COD does not declare it at all. A query must map each of these to the same physical quantity before it can be written. The second is coverage. Of the six sources, all declare a space-group number, five a band gap, four a formation energy, three an energy above the convex hull and only one a dielectric constant. A quantity that a source does not declare is absent, not zero, and it must not enter a comparison as if it were a value.

Units and quantities add a third layer. Formation energy per atom and energy above the hull are both expressed in eV per atom, yet they answer different questions: the first compares a compound with its elements, the second with the competing phases at the same composition, and both depend on each database’s reference states and corrections. A unit check alone therefore cannot establish that two values may be compared.

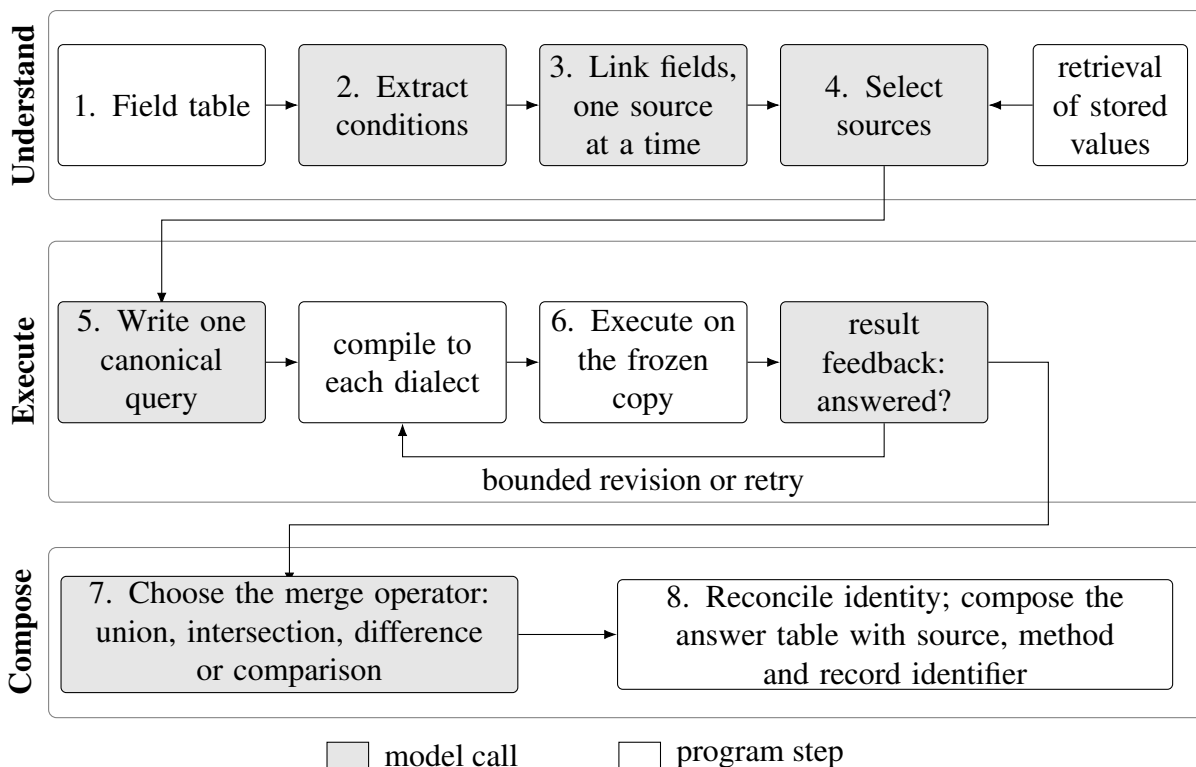


Figure 1: The eight steps of the current MatQuery pipeline. Shaded boxes are model calls; white boxes are program steps that check, execute or assemble.

2.4 What real questions look like

To see whether cross-database questions occur in practice, the team collected requests from official documentation, community forums and the screening criteria of research papers. Three examples show the range. The Materials Project documentation asks for “band gaps for all materials containing only Si and O”. A forum user asks how to obtain all available calculated band gaps of MoS₂ with the $P6_3/mmc$ space group in the Materials Project. A screening study removes “all the entries from the dataset that have energy above hull >2 eV/atom”. The requests are short and precise to their authors. They also show that real requirements are not always confined to one database: a user who wants every available calculated value, or a structure confirmed by experiment, needs evidence from several sources at once.

3 THE MATQUERY PIPELINE

3.1 Design principle

MatQuery turns a materials question into queries that run on several databases and returns a result table in which every value carries its source. The work is divided between two kinds of step (Figure 1). Model calls interpret the question, choose databases, match fields, write the query and judge the result. Program steps check that each proposal is legal, execute queries on the frozen copies, recognize the same material across databases and assemble the answer. A model call may be wrong, so its output is never executed or merged without a program step that checks it first.

3.2 Understanding the request

Step 1: field table. A program merges the fields declared by the six databases into one catalog. For each field it records the meaning, the unit and physical dimension, a plausible physical

range, whether a calculation method must be stated with it, which databases hold it and which of them allow it to be used as a filter. The catalog is built once per snapshot set and is the reference against which every later proposal is checked.

Step 2: extracting the conditions. One model call reads the question together with the catalog and produces a structured query specification: the target, the property conditions with their operators, values and units, composition and structure constraints, the requested operation and any sources the user named. The specification uses the intermediate representation described in Section 3.5, so that what the user asked for is separated from how any one database expresses it. A retrieval step gathers supporting evidence from the actual columns and stored values, such as formulas, element sets and space groups that occur in the data.

Step 3: linking fields one source at a time. For each candidate database, a separate model call receives only that database’s fields and selects those the specification needs. Presenting the fields of all six sources at once dilutes the model’s attention and invites a column name from one database to be used in another. Linking one source at a time keeps each choice clean and lets each source report its fields under its own names.

Step 4: selecting the sources. A model call proposes which databases are needed and gives a reason for each. A program step then checks coverage: every selected database must hold the fields that its part of the request uses, and every required property must be covered by at least one selected database. A selection that fails the check is rejected, not executed.

3.3 Executing the query

Step 5: one canonical query, compiled to each dialect. The model writes a single query in a database-independent form. A compiler translates it step by step into each target database’s executable statement on the local copy. When compilation fails, for example because of a syntax error or a field that does not exist in that database, the specific error is returned to the model, which may revise the query a bounded number of times. Revision can be switched off for ablation.

Step 6: execution with feedback. The compiled query runs on the frozen copy. The program returns the execution status, the number of rows, the first rows and the result columns to the model, which judges whether the query has actually answered the question. If it has not, the model may revise the query within the same bounded budget. Execution feedback can also be switched off.

3.4 Composing the answer

For a request that spans several databases, each database first returns its own part of the result. Writing q_s for the query branch sent to source s , D_s for its frozen copy and ρ for the matching policy, the prototype computes

$$R_s = q_s(D_s) \quad (s \in S), \quad \mathcal{C} = \text{reconcile} \left(\bigcup_{s \in S} R_s; \rho \right), \quad Y = \text{op}(\mathcal{C}, S), \quad (1)$$

where $\mathcal{C}$ is a set of groups of records judged to describe the same material and op is the requested operation. In *Step 7* a model call chooses op : a union keeps the groups contributed by any branch, an intersection keeps groups present in every required source, a difference keeps groups present only in the first, and a comparison keeps the grouped records side by side. In *Step 8* a program performs the reconciliation and builds the final table. The plan is executed as a directed acyclic graph of query nodes, a reconciliation node and an output node. Equation (1) describes record integration at the application layer; it is not a relational join across remote databases.

Table 2: Principal components of the intermediate representation.

Component	Information represented	Example
Target and intent	material, structure or calculation record; filter, rank or aggregate	summarize band gaps by crystal system
Property predicates	property, operator, value or bounds, unit, method, conditions	band gap between 1.1 and 1.7 eV
Composition	formula, required elements, excluded elements	halides without Pb
Structure	crystal system, space group, lattice lengths, prototype	space group $P6_3/mmc$
Answer operation	projection, sort, limit, aggregate functions, grouping	top seven records by band gap
Route and provenance	database, snapshot, native representation	bind a branch to <code>jarvis-hard</code>

Reconciliation uses the evidence the records carry: normalized formulas, element sets, space groups, lattice parameters and, where available, structure fingerprints. In composition mode, records are grouped when formula and element set agree. In structural mode, structural agreement is also required. The rule that produced each group is kept with the result, so that a grouping by composition can always be told apart from one supported by structure. The final table keeps the database name, upstream record identifier, snapshot, method string, data origin, source URL and value for every entry. Values from different databases are shown side by side and are never averaged automatically, and recorded issues concerning units, missing method information or reconciliation are attached for the reader to inspect.

3.5 The intermediate representation

The query representation separates the scientific request from the syntax of any one source (Table 2). A property constraint records the property, operator, value or range, unit, method and conditions. Composition and structure constraints describe the material, and projection, ordering, aggregation and route information describe how the answer is assembled. Deterministic validation resolves property aliases against the catalog, converts supported units to canonical units and identifies conditions that must be stated. The representation can also express a clarification requirement and accept a structured answer, which is the interface needed for experiments in which the system asks the user to settle part of a request.

4 BENCHMARK CONSTRUCTION

4.1 Approach

To evaluate natural-language querying over several materials databases with verified answers, MatQuery constructs a benchmark on its frozen snapshots. The approach reverses the usual order. Instead of writing questions and then searching for their answers, the generator first fixes a reference query that executes correctly on the frozen copies, and only then asks a model to write the question a researcher would ask. The question is accepted only if it can be read back into the same query and passes a review of fidelity and naturalness. Every accepted sample therefore has a reference answer that has been computed, not merely asserted.

4.2 Six stages

Capability pre-check. For each coverage slot, a combination of databases, properties and operations, the generator first confirms that the required fields exist and can be filtered in the relevant databases. For slots that span databases, it also confirms that the databases share materials that can be matched. A slot that fails stops here, and a capability report records the outcome.

Reference pre-check. The generator then collects enough candidate reference queries for each slot, each of which must actually run. It draws a few more candidates than needed and filters them against a set of exclusion fingerprints, so that a query meaning already used elsewhere is not produced again.

Reference synthesis. Each candidate is executed on the snapshots. A candidate is discarded if execution fails or if its result does not meet the slot’s coverage requirement, for example a comparison that returns records from only one database.

Verbalization. A model receives a reader-facing description of the reference query, its research goal and its scientific conditions, with property names, comparison, values and units in natural language, together with a style instruction. It writes one question per style. The current generator offers seven styles: explicit expert, natural scholarly, concise, colloquial, incomplete, ambiguous and multilingual. The incomplete and ambiguous styles deliberately leave out or blur part of the request and provide material for clarification experiments.

Back-translation. A separate model call reads the question back into a structured specification, and a program compares it field by field with the reference. Any mismatch rejects the question.

Alignment review. For questions that pass the field comparison, another model call acts as a reviewer. It judges whether the meaning is preserved, whether the request is scientifically sound, and whether the parts meant to be vague are actually vague. It also gives a confidence and a naturalness score, and thresholds on these decide acceptance.

4.3 Two generations of the benchmark

The development set used for the preliminary results in Section 6 contains 800 complete base questions, 20 in each of 40 coverage slots, of which 460 address one database and 340 several. It also contains 120 incomplete variants linked to their base questions, giving 920 question versions in all. Of these, 447 were produced by a controlled template renderer and 473 by model-based verbalization. Table 3 shows the operations it covers.

The current generator implements the six stages above. Its smoke-test profile, `validation_small1`, is summarized in Table 4. It contains 30 samples, one per coverage slot without repetition, of which 25 address one database and 5 several; 28 are marked hard and 2 easy. The 30 samples have 196 question variants across the seven styles. The profile records database counts of 11 for the Materials Project, 4 each for AFLOW, JARVIS-DFT, NOMAD and COD, and 3 for OQMD. The counting convention for the five multi-source samples is not specified, so these counts are not interpreted as database-usage totals. The profile is marked eligible for release. It exists to verify the pipeline from end to end, not to estimate accuracy.

A separate pool of demonstrations supports the few-shot baselines. Demonstrations come from an independently released, reviewed pool; each is routed to the profile of its own database, and a leakage check refuses any candidate that shares a sample identifier, a reference query or an unseen intent with the evaluation set. The sample under evaluation is always excluded from its own candidate set.

Table 3: Task forms in the 800-question development set, recomputed from the saved reference plans.

Task form	Questions	Share	Main requirement
Single-source filtering	340	42.5%	composition, structure and property predicates
Single-source ranking	40	5.0%	ordering and a result limit
Single-source aggregation	80	10.0%	summaries and groups
Multi-source comparison	180	22.5%	source-specific records side by side
Multi-source union	100	12.5%	combine contributions of branches
Multi-source intersection	40	5.0%	groups present in all required branches
Multi-source difference	20	2.5%	ordered difference between branches
Total	800	100%	40 coverage slots

Table 4: The current smoke-test profile `validation_small`.

Statistic	Value
Samples	30
Coverage slots	30 (one sample per slot, no repetition)
Single-source / multi-source	25 / 5
Difficulty (hard / easy)	28 / 2
Question variants	196
Available question styles	7 (expert explicit, scholarly natural, concise, colloquial, incomplete, ambiguous, multilingual)
Recorded database counts	Materials Project 11; AFLOW 4; JARVIS-DFT 4; NOMAD 4; COD 4; OQMD 3
Release status	eligible for release

5 EVALUATION HARNESS

5.1 One frozen basis for every system

The harness has three stages (Figure 2). It first prepares the evidence: the snapshots and the generated questions are hashed and pinned, and every run records which versions it used. It then runs each system on that identical basis. Finally it scores every prediction against the frozen reference with the same set of metrics and renders the tables from the stored records. A comparability gate allows two runs to be compared only when their basis is identical, so a difference in a table cannot come from a change in the data or the questions.

5.2 Systems compared

Direct prompting is the simplest baseline. Without examples, the model reads the published capability catalog of the six databases, chooses databases and writes one SQLite query, which the program executes on the frozen copies while keeping an auditable record. The one-shot and few-shot variants add exactly one, or by default three, demonstrations from the separate pool and otherwise follow the identical chain of routing, generation, execution and storage. The few-

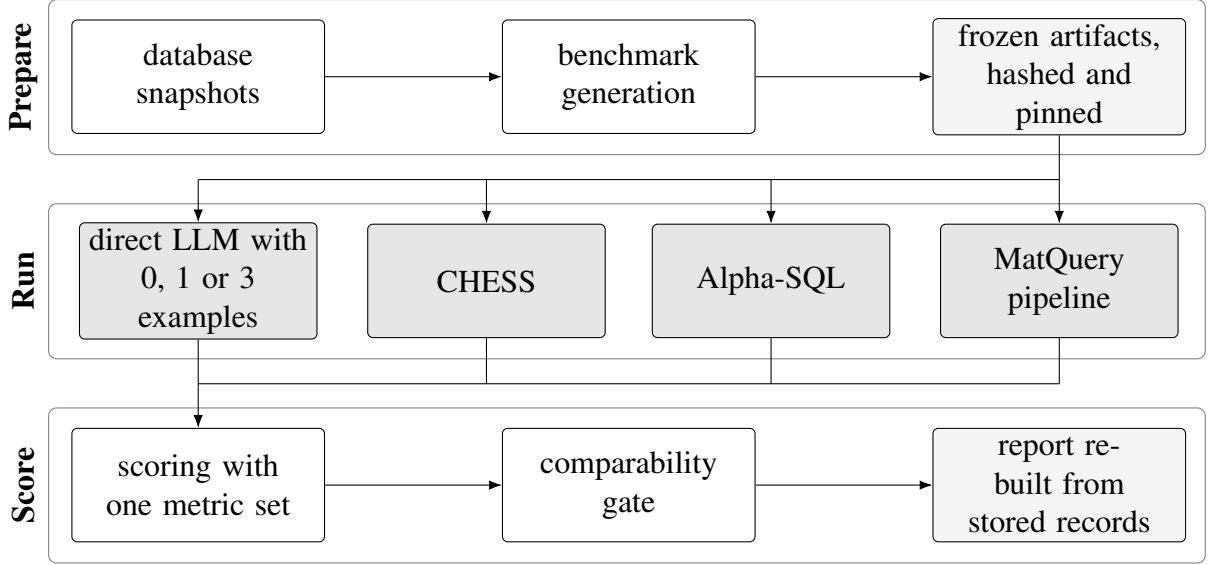


Figure 2: The evaluation harness. Shaded boxes call a language model; all systems read the same pinned snapshots and questions.

shot variant may write a separate branch query for each routed database, with one candidate and no revision per branch.

CHES [19] decomposes text-to-SQL into information retrieval, schema selection, candidate generation and natural-language unit tests. The harness runs the authors’ official implementation: it prepares input in the format CHES expects, applies compatibility patches in a disposable copy, lets CHES solve its questions with the same language model, and replays the returned queries on the frozen copies for scoring.

Alpha-SQL [16] searches over reasoning actions with Monte Carlo tree search and execution rewards. The harness runs the published source unchanged: it exports the questions as a BIRD-compatible batch, runs the upstream scripts in an isolated runtime, imports the predicted queries and replays them on the frozen copies. Its search makes it the slowest system in the harness.

Both external systems were designed for one relational database at a time. Running them on the same basis shows how far a strong single-database method goes on cross-database materials questions and compares MatQuery with two established alternatives as well as its own variants.

5.3 Metrics

The primary metric is execution accuracy. For a sample set $\mathcal{S}$, system prediction $\hat{y}_i$, reference query y_i and database D_i ,

$$\text{EX} = \frac{1}{|\mathcal{S}|} \sum_{i \in \mathcal{S}} \mathbf{1}[\text{Exec}(\hat{y}_i, D_i) \equiv \text{Exec}(y_i, D_i)], \quad (2)$$

where $\equiv$ denotes equivalence of result sets under the task’s comparison rule: ordered record identifiers for ranking, matching hashes and columns for aggregates, and record-identifier sets otherwise. The second metric is routing accuracy, the share of samples in a set $\mathcal{R}$ for which the selected set of databases equals the reference set,

$$\text{Route} = \frac{1}{|\mathcal{R}|} \sum_{i \in \mathcal{R}} \mathbf{1}[\hat{\mathcal{D}}_i = \mathcal{D}_i^{\text{gold}}]. \quad (3)$$

Exact agreement of the whole set matters because omitting one database can change a cross-database answer even when most individual choices are right. Each prediction record also stores

Table 5: Ablation switches of the current pipeline.

Switch	Default	Effect
MATQUERY_SQL_REVISION_LIMIT	2	maximum revisions after a compilation failure; 0 gives a single attempt
MATQUERY_RESULT_FEEDBACK_LIMIT	2	maximum rounds of execution feedback; 0 disables it
MATQUERY_RESULT_FEEDBACK_SAMPLE_ROWS	5	result rows shown to the model
MATQUERY_SCHEMA_LINK_PER_DATABASE	true	link fields one database at a time; false links all six together
MATQUERY_IDENTITY_RECONCILIATION	true	group the same material across databases; false disables grouping

the query, the returned rows, the tokens used and the cost, so that accuracy can be reported against cost.

Neither metric measures scientific validity. A result set equal to the reference shows agreement under the saved scoring rule. It does not show that the methods behind the values are compatible or that grouped records share a crystal structure. Those questions are examined separately (Section 6.3).

5.4 Ablation switches

The pipeline exposes switches that turn individual mechanisms on and off, so that their effect can be measured on the same basis (Table 5).

5.5 Computing environment

The experiments reported here were run on a single laptop: an ASUS TX Air FA401KM with Windows 11 (build 26200), an AMD Ryzen AI 7 H 350 processor (8 cores, 16 threads), 32 GiB of memory, an NVIDIA GeForce RTX 5060 Laptop GPU with 8 GiB of memory (driver 577.05, CUDA 12.9), a 1 TB NVMe drive, CPython 3.12.13 and uv 0.12.21. The dialogue model runs on a remote service; the laptop performs orchestration, query execution and local embedding inference, and the resident embedding model occupies 7,579 MiB of GPU memory. A complete harness, including six database copies, therefore fits on one laptop, although no part of the current pipeline yet runs its dialogue model locally.

6 PRELIMINARY RESULTS

This section reports the preliminary results. They were obtained on the 800-question development set and the snapshot collection described in Sections 2 and 4, using direct generation and separate pilot configurations rather than the complete pipeline of Section 3. The questions have not yet been reviewed by independent experts, so the results describe behavior on this development resource.

6.1 Direct query generation on 800 questions

The main runs used a remote model through the endpoint alias `deepseek-flash`, with either no demonstrations or three. Table 6 gives the results. With three demonstrations, strict execution accuracy, which requires successful execution and a matching result hash, rose from 452 to 659 of 800 questions. The exact source set was chosen correctly in about 90% of questions in both

Table 6: Direct-generation results on the same 800 complete questions (denominators 800).

Measure	No demonstrations	Three demonstrations
Execution success	515 (64.4%)	675 (84.4%)
Strict execution accuracy	452 (56.5%)	659 (82.4%)
Result equivalence	459 (57.4%)	666 (83.3%)
Exact source-set match	717 (89.6%)	718 (89.8%)
Strict accuracy, single-source (of 460)	262 (57.0%)	387 (84.1%)
Strict accuracy, multi-source (of 340)	190 (55.9%)	272 (80.0%)

runs. Of the 258 questions that became correct only with demonstrations, 216 had the correct source set in both runs, which places most of the gain in query construction rather than in source choice.

The gain was uneven across operations. Aggregation rose from 20 to 75 of 80 questions and cross-source comparison from 83 to 144 of 180, while ranking did not improve (20 and 19 of 40). Demonstrations and evaluation questions share task families, so these figures describe transfer within the development set and not to new kinds of request.

6.2 Inexpensive source scoring

A separate pilot posed one yes-or-no question per database for each of the 920 question versions, 5,520 decisions in all, and read the answer probability without generating text. Exact agreement with the reference source set was 680 of 800 on the complete questions (85.0%) and 800 of 920 across all versions (87.0%). The area under the ROC curve for single decisions was 0.995, and after temperature scaling [9] fitted in two folds grouped by base question, the expected calibration error was 0.0125 (0.0423 before scaling). The recorded provider cost of all 5,520 decisions was US\$0.032; six decisions shared one request with a median duration of 383 ms, and a decision used on average 139 input tokens. The pilot shows that such a decision can be made cheaply and with well-calibrated probabilities. On complete questions, however, its exact-set agreement was below that of the generative pipeline (85.0% against about 90%), so its case rests on cost and calibration, not on accuracy.

6.3 Cross-source audit

A deterministic audit of the six snapshots, independent of any language model, measured how much values disagree for records grouped by composition. Formula normalization produced 5,732 clusters containing records from at least two sources, each with at most one record per source. In 1,036 of the 3,275 clusters with at least two band-gap values (31.6%), the band gaps differed by more than 0.5 eV. The share was 175 of 740 (23.6%) when only records reporting the same space group were compared. Formation energies differed by more than 0.1 eV per atom in 55.7% of eligible clusters, and the stored hull-related values by more than 0.05 eV per atom in 79.0%. Disagreement of this kind has been documented for databases that use similar methods [10]. Every cluster contained more than one recorded method string, but the strings mean different things: OQMD records labels such as `static`, NOMAD records program names such as `VASP`, and JARVIS-DFT records the functional. String inequality is therefore heterogeneity in metadata, not proof of incompatible methods.

A single traced case shows why identity matters. One SiO_2 cluster contains records from AFLOW, JARVIS-DFT, the Materials Project and OQMD with band gaps of 0.000, 5.469, 5.516 and 4.606 eV. The three records that store a numerical space group report three different ones

Table 7: The 50-session pilot. “Valid” gives the counts of the two AI reviewers; “invalid” counts answers flagged by either. Denominators are 50 except for questions.

Arm	Executed	Ref. equiv.	Valid	Invalid	Questions
A: direct, no questions	42	37	19 / 19	20	0
B0: native framework, with context	7	7	7 / 5	2	0
B: adapted, no questions	2	2	2 / 2	0	0
C: ask	16	15	15 / 14	1	77
D: probe, then ask	14	13	11 / 13	3	69
E: direct agent with ask tool	38	38	29 / 21	17	18

(70, 141 and 58). The zero is present in the AFLOW payload itself and is not a substituted missing value. The four records share a composition but not, on the evidence stored, a structure, and an interface that returned one unqualified number would hide this.

6.4 Fifty-session clarification pilot

Before any model call, 50 sessions were fixed: 20 complete requests and 30 linked incomplete variants. Executing candidate values on the snapshots labelled 15 omitted slots as changing the result, 20 as not changing it, and 10 as already settled by the text. Six arms were compared on the same sessions (Table 7). Validity was labelled by two separately blinded AI reviewers and then checked by team members with materials-science backgrounds; independent expert adjudication has not taken place.

The pilot did not support the probe-and-ask design in its tested form. Arm D produced 11 answers rated valid by both reviewers, with an exact 95% interval of 11.5–36.0%, and failed its pre-specified gate of at least 25 valid and at most 2 invalid answers. Most of its failures (35 of 39) were failures to generate, compile or execute a plan, which point to the shared three-example adaptation: it executed in 2 of 50 sessions where direct generation executed in 42. The questions were also poorly targeted: 65 of the 77 questions asked by arm C and 57 of the 69 asked by arm D did not concern an omitted slot that changed the result. The direct arms, by contrast, returned an invalid answer in 17 to 20 of 50 sessions. Arm E is descriptive only, because the scripted replies missed or under-answered ten of its questions. The two reviewers agreed on 273 of 300 labels (Cohen’s $\kappa = 0.836$).

6.5 Three-hundred-pair structure audit

A sample of 300 record pairs was frozen: 150 pairs linked by the composition matcher, 100 unlinked pairs with the same reported space group and 50 unlinked pairs with different space groups. Only JARVIS-DFT kept full atomic coordinates in the frozen payloads, so on the snapshots alone every pair was “cannot tell”. Retrieving public structures by the saved record identifiers supplied both structures for 115 pairs. Automatic labels from the pymatgen structure matcher [17] are given in Table 8. Among the linked pairs that could be resolved, 13 were the same structure and 26 different.

Two AI reviews of 78 selected pairs agreed on 71 ($\kappa = 0.849$), and the labels were checked by team members with materials-science backgrounds. The resulting estimates of linkage quality are uncertain. The design-weighted precision of composition links is 33.1% (95% interval 19.6–47.8%) on the resolved pairs, but because structures were missing in a non-random way the partial-identification range is 8.5–82.8%. Only two pairs judged to be the same structure

Table 8: Automatic structure labels for the 300 sampled pairs (pymatgen StructureMatcher with volume scaling, after supplementary retrieval). The labels are a review aid, not ground truth.

Stratum	Pairs	Same	Different	Cannot tell
Linked by composition	150	13	26	111
Unlinked, same space group	100	35	17	48
Unlinked, different space group	50	0	24	26
Total	300	48	67	185

had two band-gap values, so the audit cannot yet say how large the disagreement is between records known to describe the same crystal.

6.6 What the preliminary results establish

Three conclusions survive the limits of these data. First, a substantial share of cross-database requests can be turned into queries that return the reference result, but this establishes query correctness and not scientific comparability. Second, values grouped by composition disagree often enough, and composition links are unreliable enough, that identity must be judged by structure before values are compared. Third, the simple probe-and-ask design failed in its tested form, mainly because the adapted generation path broke and the questions asked were poorly targeted, not because asking is unnecessary: the arms that asked nothing returned invalid answers in up to 40% of sessions. These findings shaped the pipeline of Section 3, which links fields one database at a time, checks coverage before execution, and revises queries with bounded feedback.

7 CURRENT STATUS AND NEXT STEPS

7.1 Where the work stands

The complete pipeline, the benchmark generator and the harness with all baselines are implemented and in smoke testing on the `validation_small` profile. Three steps remain before results can be reported: completing the smoke-test validation, running the large-sample experiments together with the ablations of Table 5, and evaluating and assembling all results. Alpha-SQL is the slowest system to run because of its tree search. When these runs are complete, a revision of this report will give their results alongside the preliminary results of Section 6.

7.2 Open items

Several items must be settled before the benchmark can support confirmatory claims. The questions have passed automatic checks and model-based review but no independent expert review, which is the first step planned with materials scientists. Structures must be re-acquired for all six sources, since only JARVIS-DFT kept coordinates in the snapshots used for the preliminary results. The demonstration pool and the evaluation set share task families, so a held-out split by family is needed to measure transfer. Finally, the preliminary results were not produced by the complete pipeline, so they cannot be read as its performance.

7.3 Research directions supported by the prototype

The components described in this report support four lines of further work. First, the field catalog and the execution of candidate values on the snapshots make it possible to decide, from the

data, whether an unstated part of a request actually changes the answer, and therefore whether it is worth asking the user about it. Second, the per-prediction records of tokens, model calls, executions and cost make it possible to study how a fixed budget of computation and user interaction should be spent within a session. Third, the typed steps of the pipeline (condition extraction, field linking, source selection and the choice of merge operator), together with traces verified by execution, form narrow decisions and training data on which smaller, locally deployed models can be adapted and measured. Fourth, the compiler, bounded revision and execution feedback are the starting point of a local runtime in which deterministic checks compensate for a weaker model. The preliminary pilots set out what such work must improve on: clarification questions that target the parts of a request that change the result, and a generation path that does not break when it is adapted.

8 LIMITS OF THE EVIDENCE AND REPRODUCIBILITY

The evidence in this report has a defined scope. The data are selected frozen subsets of six sources, not samples of the complete databases. The questions are generated and checked automatically, and independent expert review is pending. The pilot labels were produced by AI reviewers and checked by team members with materials-science backgrounds, which is an internal review rather than independent adjudication. Supplementary structures were retrieved after the original snapshot freeze. Demonstrations and evaluation questions share task families. The model endpoint was named by an alias that does not identify an immutable model revision, so the saved outputs, not the alias, are the definitive record of each run. All figures therefore describe behavior on these resources and are not general accuracy estimates for scientific queries.

Every reported figure is traceable to stored artifacts: snapshot manifests with file digests, the benchmark manifest with its samples and reference plans, per-question prediction records with queries, rows, tokens and cost, decision scores, and the deterministic audit report. The harness rebuilds its tables from these records, and the comparability gate prevents results obtained on different bases from appearing side by side. Release of the code, snapshot rebuild scripts and benchmark is planned alongside the full evaluation results. The release licence and distribution location remain to be determined.

9 CONCLUSION

MatQuery provides an execution-grounded infrastructure for natural-language querying over six heterogeneous materials databases. Its frozen snapshots, declared-field catalog, canonical query representation, source-specific adapters and evaluation harness establish a common basis for studying query generation, source selection and evidence-preserving answer composition. The preliminary development-set results show that demonstrations can improve strict execution accuracy from 56.5% to 82.4%, while the cross-source audits show why execution correctness must be distinguished from scientific comparability. Composition alone does not establish structural identity, and the tested clarification policy did not meet its pre-specified validity gate.

The complete eight-step pipeline is implemented but remains in smoke testing. Its end-to-end accuracy and ablation results have not yet been established. The next phase therefore requires full-scale evaluation on a fixed basis, independent expert review, improved structural coverage and tests of transfer beyond the shared development task families. Until those checks are complete, the report’s findings should be read as evidence about the described prototype and development resources, with the limitations stated in Section 8.

REFERENCES

- [1] C. W. Andersen et al. OPTIMADE, an API for Exchanging Materials Data. *Sci. Data* 8:217, 2021.
- [2] Y. Chiang et al. LLaMP: Large Language Model Made Powerful for High-fidelity Materials Knowledge Retrieval. In *EMNLP*, pp. 25189–25221, 2025.
- [3] K. Choudhary et al. The Joint Automated Repository for Various Integrated Simulations (JARVIS) for Data-Driven Materials Design. *npj Comput. Mater.* 6:173, 2020.
- [4] S. Curtarolo et al. AFLOW: An Automatic Framework for High-Throughput Materials Discovery. *Comput. Mater. Sci.* 58:218–226, 2012.
- [5] C. Draxl and M. Scheffler. The NOMAD Laboratory: From Data Sharing to Artificial Intelligence. *J. Phys. Mater.* 2(3):036001, 2019.
- [6] M. L. Evans et al. Developments and Applications of the OPTIMADE API for Materials Discovery, Design, and Data Exchange. *Digital Discovery* 3(8):1509–1533, 2024.
- [7] D. Gao et al. Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation. *Proc. VLDB Endow.* 17(5):1132–1145, 2024.
- [8] S. Gražulis et al. Crystallography Open Database – An Open-Access Collection of Crystal Structures. *J. Appl. Crystallogr.* 42(4):726–729, 2009.
- [9] C. Guo et al. On Calibration of Modern Neural Networks. In *ICML, PMLR 70*, pp. 1321–1330, 2017.
- [10] V. I. Hegde et al. Quantifying Uncertainty in High-Throughput Density Functional Theory: A Comparison of AFLOW, Materials Project, and OQMD. *Phys. Rev. Mater.* 7:053805, 2023.
- [11] A. Jain et al. Commentary: The Materials Project: A Materials Genome Approach to Accelerating Materials Innovation. *APL Mater.* 1(1):011002, 2013.
- [12] Y. Kang and J. Kim. ChatMOF: An Artificial Intelligence System for Predicting and Generating Metal-Organic Frameworks Using Large Language Models. *Nat. Commun.* 15:4705, 2024.
- [13] S. Kirklin et al. The Open Quantum Materials Database (OQMD): Assessing the Accuracy of DFT Formation Energies. *npj Comput. Mater.* 1:15010, 2015.
- [14] F. Li and H. V. Jagadish. Constructing an Interactive Natural Language Interface for Relational Databases. *Proc. VLDB Endow.* 8(1):73–84, 2014.
- [15] J. Li et al. Can LLM Already Serve as A Database Interface? A BIG Bench for Large-Scale Database Grounded Text-to-SQLs. In *NeurIPS*, 2023.
- [16] B. Li et al. Alpha-SQL: Zero-Shot Text-to-SQL using Monte Carlo Tree Search. In *ICML, PMLR 267*, pp. 36810–36830, 2025.
- [17] S. P. Ong et al. Python Materials Genomics (pymatgen): A Robust, Open-Source Python Library for Materials Analysis. *Comput. Mater. Sci.* 68:314–319, 2013.
- [18] M. Pourreza and D. Rafiei. DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction. In *NeurIPS*, pp. 36339–36348, 2023.
- [19] S. Talaei et al. CHESS: Contextual Harnessing for Efficient SQL Synthesis. *arXiv:2405.16755*, 2024.
- [20] T. Yu et al. Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. In *EMNLP*, pp. 3911–3921, 2018.